

Ellenfél alapú támadások (FGSM, PGD)
implementációja
konvolúciós neurális hálózatokhoz
interaktív tesztkörnyezetben

Gabrity Gábor
Neptun kód: D9V09Z

2026. szeptember 8.

Nyilatkozat

Alulírott Gabrity Gábor (Neptun kód: D9V09Z), az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférést a benyújtástól számított két évig nem kérem.

Budapest, 2026. szeptember 8.

Gabrity Gábor s.k.
hallgató aláírása

Kivonat

A mesterséges intelligencia rohamos fejlődésével párhuzamosan kritikus kérdésé vált a modellek biztonsága és megbízhatósága. Kutatások kimutatták, hogy a modern konvolúciós neurális hálózati architektúrák sebezhetőek az ellenfél alapú támadásokkal szemben, mint például az FGSM (Fast Gradient Sign Method) és a PGD (Projected Gradient Descent). Ezek az algoritmusok a bemeneti adatot képesek olyan módon – az emberi szem számára szinte észrevétlenül – módosítani, hogy a nagy megbízhatóságú modelleket is megtévesztik.

Szakdolgozatom célja egy olyan interaktív tesztkörnyezet fejlesztése, amely grafikus felhasználói felületen keresztül teszi lehetővé ezen biztonsági rések vizuális elemzését. A projekt fókuszában az AI Red Teaming módszertan áll: a fejlesztő vagy biztonsági szakember a szoftver segítségével szisztematikusan „támadja” a választott képfelismerő modellt, hogy feltárja annak gyenge pontjait.

A keretrendszer több elismert támadási algoritmust implementál, köztük a gyors, egy lépéses FGSM-et és a robusztusabb, iteratív PGD-t. A felhasználó számára lehetőség nyílik mind általános tévesztés (non-targeted), mind specifikus célosztályra irányuló (targeted) támadások végrehajtására, miközben interaktív módon, csúszkák segítségével szabályozhatja a perturbáció mértékét és az iterációk számát.

Az alkalmazás hármas vizuális megjelenítést alkalmaz: megjeleníti az eredeti képet, a generált zajt (perturbációt) és a végső, támadott képet, miközben mutatja a modell predikcióinak változását. A rendszer kiegészülhet egy modellvédelmi modulal is, amely különböző képfeldolgozási szűrők – például homályosítás – segítségével próbálja megsemmisíteni a beágyazott zajt, mérve ezzel a modell védekezőképességét.

Tartalomjegyzék

1. Rövidítés- és jelölésjegyzék	4
2. Bevezetés	5
2.1. Motiváció és a témaválasztás indoklása	5
2.2. A megoldandó feladat rövid, közérthető leírása	5
2.3. A dolgozat célja és hozzájárulásai	5
2.4. A dolgozat felépítése	5
3. Felhasználói dokumentáció	6
3.1. A megoldott probléma rövid megfogalmazása	6
3.2. A felhasznált módszerek rövid áttekintése	6
3.2.1. FGSM (Fast Gradient Sign Method) röviden	6
3.2.2. PGD (Projected Gradient Descent) röviden	6
3.2.3. Non-targeted és targeted támadási módok	6
3.3. Rendszerkövetelmények	6
3.4. Telepítés és első indítás	6
3.5. A grafikus felhasználói felület bemutatása	6
3.5.1. Modell kiválasztása és kép betöltése	6
3.5.2. Támadási paraméterek beállítása (ϵ , iterációszám, célosztály)	6
3.5.3. Az eredeti, a perturbáció és a támadott kép megjelenítése	6
3.5.4. Predikciók és konfidenciák leolvasása	6
3.6. Tipikus felhasználási forgatókönyvek	6
3.6.1. Non-targeted támadás végrehajtása	6
3.6.2. Targeted támadás egy adott célosztályra	6
3.6.3. Védekezési modul használata (szűrők alkalmazása)	6
3.7. Hibaüzenetek és gyakori problémák	6
4. Fejlesztői dokumentáció	7
4.1. A probléma részletes specifikációja	7
4.1.1. Funkcionális követelmények	7
4.1.2. Nem funkcionális követelmények	7
4.1.3. Használati esetek	7
4.2. Tervezési döntések és indoklásuk	7
4.2.1. Programozási nyelv és keretrendszer kiválasztása	7
4.2.2. Modellarchitektúra és előtanított modellek választása	7
4.2.3. FGSM és PGD támadások kiválasztásának indoklása	7
4.2.4. GUI technológia és interakciós modell kiválasztása	7
4.2.5. Védekezési eljárások kiválasztásának indoklása	7

4.3.	Elméleti háttér és a használt fogalmak definíciója	7
4.3.1.	Konvolúciós neurális hálózatok rövid áttekintése	7
4.3.2.	Veszteségfüggvény, gradiens és backpropagation	7
4.3.3.	Perturbáció, ℓ_∞ norma, ellenfél alapú példa	7
4.3.4.	AI Red Teaming módszertan	7
4.4.	Ellenfél alapú támadások áttekintése	7
4.5.	Kapcsolódó munkák és további támadási módszerek	7
4.5.1.	Iteratív FGSM (BIM) és Momentum-alapú változatok	7
4.5.2.	Carlini–Wagner és DeepFool támadások	7
4.5.3.	Transzferálhatóság és fekete dobozos támadások	7
4.6.	Védekezési stratégiák áttekintése	7
4.6.1.	Ellenfél alapú tanítás (adversarial training)	7
4.6.2.	Bemenet-előfeldolgozáson alapuló védekezés (szűrők, homályosítás)	7
4.6.3.	Detekció-alapú megközelítések	7
4.7.	A rendszer architektúrája	7
4.7.1.	Komponensek és modulfelbontás	7
4.7.2.	Adatfolyam a betöltéstől a támadott képig	7
4.7.3.	A GUI és a támadási motor kapcsolata	7
4.8.	A felhasznált technológiák	7
4.8.1.	Python és tudományos csomagok (NumPy, Matplotlib)	7
4.8.2.	Mélytanulási keretrendszer (PyTorch/TensorFlow)	7
4.8.3.	Grafikus felület (pl. Tkinter, PyQt vagy Streamlit)	7
4.9.	Az egyes komponensek megvalósítása	7
4.9.1.	Modellbetöltés és előfeldolgozás	7
4.9.2.	FGSM támadás implementációja	7
4.9.3.	PGD támadás implementációja	7
4.9.4.	Targeted és non-targeted mód kezelése	7
4.9.5.	Perturbáció megjelenítése és képnormalizálás	7
4.9.6.	Interaktív paraméterezés (csúszkák, célosztály választás)	7
4.9.7.	Védekezési modul (képfeldolgozási szűrők)	7
5.	Tesztelés	8
5.1.	Tesztelési terv	8
5.1.1.	Tesztelési célok és lefedettség	8
5.1.2.	Teszt adatkészletek és modellek kiválasztása	8
5.1.3.	Mérési metrikák (támadási sikerráta, ℓ_∞ eltérés, konfidencia)	8
5.2.	Egységtesztetek	9
5.2.1.	predict() és a diszpécser (attack()) tesztjei	9
5.2.2.	FGSM modul tesztjei (_fgsm)	10

5.2.3.	PGD modul tesztjei (<code>_pgd</code>)	11
5.3.	Integrációs tesztek	13
5.3.1.	GUI és támadási motor összekötése	13
5.3.2.	Modellbetöltés és teljes támadási folyamat	13
5.4.	Funkcionális tesztek és felhasználói validáció	13
5.5.	Teszteredmények és értelmezésük	15
5.5.1.	FGSM eredmények különböző ϵ értékekre	15
5.5.2.	PGD eredmények különböző iterációszámokra	15
5.5.3.	Targeted vs. non-targeted összehasonlítás	15
5.5.4.	A védekezési modul hatásának mérése	15
5.6.	A tesztelés korlátai	15
6.	Összefoglalás	16
6.1.	Elért eredmények	16
6.2.	Következtetések	16
6.3.	Továbbfejlesztési lehetőségek	16
	Mellékletek	17

1. Rövidítés- és jelölésjegyzék

2. Bevezetés

2.1. Motiváció és a témaválasztás indoklása

2.2. A megoldandó feladat rövid, közérthető leírása

2.3. A dolgozat célja és hozzájárulásai

2.4. A dolgozat felépítése

3. Felhasználói dokumentáció

3.1. A megoldott probléma rövid megfogalmazása

3.2. A felhasznált módszerek rövid áttekintése

3.2.1. FGSM (Fast Gradient Sign Method) röviden

3.2.2. PGD (Projected Gradient Descent) röviden

3.2.3. Non-targeted és targeted támadási módok

3.3. Rendszerkövetelmények

3.4. Telepítés és első indítás

3.5. A grafikus felhasználói felület bemutatása

3.5.1. Modell kiválasztása és kép betöltése

3.5.2. Támadási paraméterek beállítása (ϵ , iterációszám, célosztály)

3.5.3. Az eredeti, a perturbáció és a támadott kép megjelenítése

3.5.4. Predikciók és konfidenciák leolvasása

3.6. Tipikus felhasználási forgatókönyvek

3.6.1. Non-targeted támadás végrehajtása

3.6.2. Targeted támadás egy adott célosztályra

3.6.3. Védekezési modul használata (szűrők alkalmazása)

3.7. Hibaüzenetek és gyakori problémák

4. Fejlesztői dokumentáció

4.1. A probléma részletes specifikációja

4.1.1. Funkcionális követelmények

4.1.2. Nem funkcionális követelmények

4.1.3. Használati esetek

4.2. Tervezési döntések és indoklásuk

4.2.1. Programozási nyelv és keretrendszer kiválasztása

4.2.2. Modellarchitektúra és előtanított modellek választása

4.2.3. FGSM és PGD támadások kiválasztásának indoklása

4.2.4. GUI technológia és interakciós modell kiválasztása

4.2.5. Védekezési eljárások kiválasztásának indoklása

4.3. Elméleti háttér és a használt fogalmak definíciója

4.3.1. Konvolúciós neurális hálózatok rövid áttekintése

4.3.2. Veszteségfüggvény, gradiens és backpropagation

4.3.3. Perturbáció, ℓ_∞ norma, ellenfél alapú példa

4.3.4. AI Red Teaming módszertan

4.4. Ellenfél alapú támadások áttekintése

4.5. Kapcsolódó munkák és további támadási módszerek

4.5.1. Iteratív FGSM (BIM) és Momentum-alapú változatok

4.5.2. Carlini–Wagner és DeepFool támadások

4.5.3. Transzferálhatóság és fekete dobozos támadások

4.6. Védekezési stratégiák áttekintése

4.6.1. Ellenfél alapú tanítás (adversarial training)

4.6.2. Bemenet-előfeldolgozáson alapuló védekezés (szűrők, homályosítás)

4.6.3. Detekció-alapú megközelítések

4.7. A rendszer architektúrája

4.7.1. Komponensek és modulfelbontás

4.7.2. Adatfolyam a betöltéstől a támadott képig

4.7.3. A GUI és a támadási motor kapcsolata

5. Tesztelés

5.1. Tesztelési terv

5.1.1. Tesztelési célok és lefedettség

A program tesztkészletének felépítésénél a szoftvertesztelési szakirodalomból ismert *teszt-piramisnak* nevezett hierarchiát követtem. Ennek alapját a gyorsabb és kevés komponenst érintő tesztek, az egységtesztek képezik, melyek a lehető legkisebb elkülöníthető egységek a többi egységtől független tesztelését valósítják meg. Futási idejük jellemzően ezredmásodperces tartományban mozog és számos esetet fednek le, mint például az inputok szélsőértékeit, hibás működést, helyes működést. Az elkészítésüknél fontos szempont a többi egységtől való függetlenség (izoláció) és az perifériáktól való függetlenség, I/O, adatbázis, hálózat, stb. Ezeknek a teszteknek az elkészítésére érdemes nagy figyelmet fordítani, mivel a szoftverek hibáinak jelentős része helyes egységtesztekkel kiküszöbölhető lehetne.

A lefedettség középpontjában az `attack_engine.py` modul áll, mivel ez valósítja meg az alkalmazás lényegi, biztonsági szempontból is kritikus logikáját (az FGSM és PGD algoritmusokat, valamint az ezeket kiszolgáló `predict()` és `attack()` függvényeket).

ezek jelen fejezetben nem kerülnek részletes bemutatásra, mivel nem az algoritmusok, hanem a körülöttük lévő „vezetékezés” helyességét igazolják.

5.1.2. Teszt adatkészletek és modellek kiválasztása

Az egységtesztek tudatosan nem a valódi, előtanított ResNet50/MobileNetV2 hálót használják. Az `attack_engine` teszteknél egy `tiny_model` fixture ad egy 5 osztályos, 8×8 pixeles bemenetet feldolgozó, egyetlen lineáris rétegből álló hálót; a `model_manager` teszteknél pedig a `models.resnet50/models.mobilenet_v2` konstruktorokat helyettesíti egy `mock.patch` által visszaadott apró `nn.Module`. Ez a döntés a tesztek sebességét (a valódi háló letöltése és felépítése másodperces nagyságrendű lenne) és izoláltságát szolgálja: a teszt kizárólag a saját kódunk logikáját (gradiensszámítás, clamp, dispatch, cache) vizsgálja, nem a harmadik féltől származó architektúra pontosságát. A valódi modellekkel való működés ellenőrzése az integrációs/GUI rétegek feladata.

Minden véletlent felhasználó fixture (pl. a bemeneti képet generáló `sample_image_tensor`) maga hívja meg a `torch.manual_seed`-et közvetlenül az adat előállítása előtt, így a determinizmus független a fixture-ök végrehajtási sorrendjétől.

5.1.3. Mérési metrikák (támadási sikerráta, ℓ_∞ eltérés, konfidencia)

Az egységtesztek szintjén nem támadási sikerrátát mérünk (ehhez sok mintás, statisztikai kiértékelés kellene), hanem az algoritmusok *tulajdonságait* és *kontraktusait* ellenőrizzük konkrét, kis méretű bemeneteken:

- az ℓ_∞ korlát ($\|\delta\|_\infty \leq \epsilon$) ténylegesen teljesül-e a perturbációra,
- a predikciós konfidencia a várt irányba mozdul-e (csökken a valódi címkénél non-targeted, nő a célosztálynál targeted esetben),
- a kimeneti tenzor a $[0, 1]$ pixeltartományban marad-e,
- az eredmény le van-e választva (`detach`) az autograd gráfról.

5.2. Egységtesztek

Az alábbiakban nem az összes egységtesztet sorolom fel – ez a `model_manager` és `utils` modulok betöltési/konverziós teszteseivel együtt túlmutatna a fejezet keretein –, hanem kizárólag a támadásokat ténylegesen *végrehajtó motor*, azaz az `attack_engine.py` modul teszteseinek logikáját mutatom be részletesen, mivel ez hordozza a dolgozat szempontjából releváns algoritmikus tartalmat. A táblázatokban minden sor egy konkrét teszt independensen ellenőrizhető állítását írja le; mivel ez egy funkcionálisan tiszta modul (a modell és a normalizációs paraméterek explicit `AttackContext`-ként érkeznek, nincs rejtett I/O), a teszteknek nem kellett mock-objektumokat használniuk.

5.2.1. `predict()` és a diszpécser (`attack()`) tesztjei

A `predict()` függvény a modell nyers kimenetét alakítja *top-k* valószínűségekkel, az `attack()` pedig a bekért módszer ("FGSM" vagy "PGD") alapján szétosztja a hívást a belső `_fgsm/_pgd` implementációk közül a megfelelőre, majd összeállítja a visszaadott `AttackResult` objektumot. Ez a réteg felelős a bemenet-validációért és az eredmény-összeállításért, magáért a perturbáció-számításért nem.

Teszteteset	Leírás
<code>test_predict_re- turns_topk_shapes</code>	Ellenőrzi, hogy <code>topk=3</code> esetén a visszaadott valószínűség- és index-tenzor alakja $(1, 3)$, azaz a batch- és k -dimenzió megfelel a kérésnek.
<code>test_predict_re- turns_valid_distribution</code>	A <code>topk=5</code> valószínűségek mindegyike a $[0, 1]$ intervallumba esik, és – mivel az 5 osztályos <code>tiny_model</code> minden osztálya lekérdezésre kerül – összegük 1,0-ra <code>allclose</code> . Ez a softmax kimenet valódi valószínűségi eloszlás voltát igazolja.
<code>test_predict_top- k_returns_highest_first</code>	A visszaadott valószínűségek csökkenő sorrendben szerepelnek (<code>probs[0, i] ≥ probs[0, i + 1]</code>), vagyis a <code>topk</code> valóban a legnagyobb konfidenciájú osztályokat adja vissza, méghozzá rendezetten.

Teszt eset	Leírás
<code>test_attack_unknown_method_raises</code>	Határeset: egy nem létező módszernév ("NEM VALID") megadásakor <code>ValueError</code> -t vár, mielőtt bármilyen gradiensszámítás történne – a bemenet-validáció a drága számítás előtt fut le.
<code>test_attack_fgsm_ignores_iterations_argument</code>	Annak ellenére, hogy a hívó <code>iterations=99</code> -et ad át, FGSM esetén a visszaadott <code>AttackResult.iterations_run</code> értéke 1 – igazolva, hogy az FGSM valóban egy lépéses, és a paramétert a diszpécser figyelmen kívül hagyja erre a módszerre.
<code>test_attack_pgd_ports_requested_iteration_count</code>	PGD esetén a kért iterációs szám (4) változatlanul megjelenik az <code>iterations_run</code> mezőben, alátámasztva, hogy a diszpécser a paramétert továbbadja a <code>_pgd</code> implementációnak.
<code>test_attack_fgsm_dispatches_only_to_fgsm</code>	<code>mock.patch(..., wraps=_fgsm)</code> és <code>wraps=_pgd</code> spy-okkal figyeli a hívásokat: <code>method="FGSM"</code> esetén <code>_fgsm.assert_called_once()</code> és <code>_pgd.assert_not_called()</code> teljesül. A <code>wraps</code> paraméter miatt a valódi implementáció is lefut, tehát az <code>AttackResult</code> összeállítása is ellenőrzött marad.
<code>test_attack_pgd_dispatches_only_to_pgd</code>	Az előző teszt tükörképe: <code>method="PGD"</code> esetén kizárólag <code>_pgd</code> hívódik meg, <code>_fgsm</code> nem. A két teszt együtt zárja ki, hogy a diszpécser valamelyik ágon „átcsorogva” mindkét algoritmust lefuttassa.

5.2.2. FGSM modul tesztjei (`_fgsm`)

Teszt eset	Leírás
<code>test_fgsm_targeted_steps_opposite_direction_of_untargeted</code>	Az <code>direction</code> paraméterrel a <code>targeted</code> és <code>untargeted</code> perturbáció pontosan ellentétes előjelű ($x_{\text{untargeted}} - x = -(x_{\text{targeted}} - x)$), ami közvetlenül a matematikai definícióból (a gradiens előjelének megfordításából) következik.

Teszt eset	Leírás
<code>test_fgsm_clamps_to_valid_pixel_range</code>	Szükséges bemenet (csupa nulla pixel) és nagy $\epsilon = 0,5$ mellett is a kimenet minden eleme a $[0, 1]$ tartományban marad – igazolva, hogy a <code>clamp</code> lépés ténylegesen érvényesül, nem csak elméletben szerepel a specifikációban.
<code>test_fgsm_result_is_detached_from_autograd</code>	A visszaadott tenzor <code>requires_grad</code> attribútuma <code>False</code> .
<code>test_fgsm_result_has_no_graph_history</code>	Szigorúbb bizonyíték az előzőnél: <code>grad_fn is None</code> és <code>is_leaf is True</code> egyszerre teljesül, ami kizárja azt az esetet is, amikor egy tenzor <code>requires_grad</code> értéke hamis, de mégis egy számítási gráf levele marad rejtett hivatkozásokkal.
<code>test_fgsm_zero_epsilon_returns_original</code>	Határeset: $\epsilon = 0$ esetén a perturbáció nulla, a kimenet <code>allclose</code> az eredeti képpel – az algoritmus degenerált esetben identitásként viselkedik.
<code>test_fgsm_untargeted_reduces_confidence_in_true_label</code>	Viselkedési bizonyíték: a <code>predict()</code> -en keresztül lekérdezett, a valódi címkéhez tartozó konfidencia a támadás <i>után</i> szigorúan kisebb, mint előtte – ez nem csak a perturbáció létezését, hanem annak tényleges hatását igazolja a modell kimenetére.
<code>test_fgsm_targeted_increases_confidence_in_target_label</code>	A <code>targeted</code> változat tükörpárja: a mesterségesen kiválasztott célosztály konfidenciája nő a támadás hatására, alátámasztva, hogy a <code>targeted</code> mód valóban a célosztály felé tereli a predikciót, nem csupán „valamerre” téveszti meg a modellt.

5.2.3. PGD modul tesztjei (`_pgd`)

Teszt eset	Leírás
<code>test_pgd_result_stays_within_epsilon_ball</code>	Mérsékelt lépésméret ($\alpha = 0,02$) mellett a végső perturbáció minden pixelre $ \delta \leq \epsilon + 10^{-6}$ (a tolerancia a lebegőpontos kerekítést kompenzálja) – ez az ℓ_∞ -gömbbe való projekció alapvető helyességét igazolja.

Teszt eset	Leírás
<code>test_pgd_hits_ball_when_step_too_large</code>	Számkorosan irreálisan nagy lépésméretet ($\alpha = 10,0 \gg \epsilon = 0,02$) ad meg, ami minden iterációban túllövést okoz; ilyenkor a projekciós <code>clamp</code> -nek kell egyedül biztosítania a korlátot, ezért a tesztben a végső $ \delta $ pontosan (nem csak $\leq$) ϵ -nak adódik minden pixelen – ez a projekció, nem a lépésméret helyes megválasztása általi korlátozottságot bizonyítja.
<code>test_pgd_clamps_to_valid_pixel_range</code>	A <code>clamp</code> pixel tartomány-korlátozás ugyanúgy teljesül PGD esetén is, mint FGSM-nél: szélsőséges (nulla) bemeneten és nagy ϵ mellett a kimenet a $[0, 1]$ tartományban marad.
<code>test_pgd_result_has_no_graph_history</code>	Ugyanaz az erős <code>detach</code> -bizonyíték (<code>grad_fn is None</code> és <code>is_leaf is True</code>), mint FGSM esetén – fontos, mert a PGD több iterációt is végez, minden lépésben új <code>requires_grad_(True)</code> hívással, így könnyebben „elszivároghatna” egy hivatkozás a gráfra, ha a ciklus végén elmaradna a leválasztás.
<code>test_pgd_zero_epsilon_returns_original</code>	Határeset: $\epsilon = 0$ esetén – annak ellenére, hogy több iteráció is lefut – a projekció minden lépésben nulla szélességű intervallumra szorítja a perturbációt, így a végeredmény megegyezik az eredeti képpel.
<code>test_pgd_untargeted_reduces_confidence_in_true_label</code>	Viselkedési bizonyíték <code>predict()</code> -en keresztül: a valódi címke konfidenciája több iteráció után is csökken – ugyanaz az elv, mint FGSM-nél, de itt azt is igazolja, hogy az iteratív frissítés nem „rontja el” a kumulatív hatást.
<code>test_pgd_targeted_increases_confidence_in_target_label</code>	A targeted PGD a célosztály konfidenciáját növeli – ugyanazon tulajdonságpár (non-targeted csökkenti/targeted növeli a releváns konfidenciát) FGSM-re és PGD-re is parametrizált formában lett tesztelve (<code>@pytest.mark.parametrize</code>), elkerülve, hogy csak az egyik algoritmusra íródjon le a viselkedés.

5.3. Integrációs tesztek

5.3.1. GUI és támadási motor összekötése

Az integrációs réteg feladata annak ellenőrzése, hogy a `src/gui/app_interface.py` eseménykezelői (pl. az `Attack` gombhoz kötött `_on_attack/_attack_worker` páros) helyesen építik-e fel az `AttackContext`-et a ténylegesen betöltött `ModelBundle`-ból, és a `attack_engine.attack()` hívás eredménye helyesen jelenik-e meg a felületen (kép- és predikció-frissítés). Ez a réteg már nem mockolja a modellt, de a valódi felhasználói interakciót (egérkattintás, csúszka-mozgatás) sem szimulálja még – azt a GUI-tesztek rétege végzi.

5.3.2. Modellbetöltés és teljes támadási folyamat

Ezen a szinten a `ModelManager` valódi (nem mockolt) `ResNet50/MobileNetV2` betöltésével, a lemezen cache-elt súlyokkal fut le a teljes folyamat – kép betöltése, előfeldolgozás, támadás, predikció –, hogy kiderüljön, a valódi architektúrák bemeneti/kimeneti alakjai (`shape`) és a normalizációs pipeline illeszkednek-e az `attack_engine` elvárásaihoz.

5.4. Funkcionális tesztek és felhasználói validáció

A grafikus felület funkcionális, „fekete doboz” tesztéseit a felhasználói forgatókönyvek alapján, *Given–When–Then* formában fogalmazom meg. Az alábbi esetek a `src/gui/app_interface.py` tényleges felhasználói felületének elemeihez (gombok, csúszkák, célosztály-kereső) igazodnak, és egyelőre kézi (manuális) végrehajtásra tervezettek.

- **Kép betöltése a „Load Image” gombbal**

AS A: felhasználó I WANT TO: egy képet betölteni a vizsgálathoz

GIVEN: az alkalmazás elindítva, még nincs kép betöltve

WHEN: a „Load Image” gombra kattint, majd egy érvényes képfájlt választ ki

THEN: az „Original” képmezőben megjelenik a beolvasott, 224×224 -re vágott kép, és alatta a top-3 predikció és konfidencia frissül

- **Kép betöltése húzd-és-ejtsd módon**

AS A: felhasználó I WANT TO: gyorsan, fájlböngésző nélkül betölteni egy képet

GIVEN: a `tkinterdnd2` csomag elérhető, az alkalmazás fut

WHEN: egy képfájl az „Original” képmezőre húz és ott elenged

THEN: ugyanaz történik, mintha a „Load Image” gombbal töltötte volna be

- **Modell váltása betöltött kép mellett**

AS A: felhasználó I WANT TO: összehasonlítani, hogy más architektúra hogyan látja a képet

GIVEN: egy kép már be van töltve, az „Original” predikciós panel ki van töltve

WHEN: a modell-választó legördülőben „MobileNetV2”-t választ „ResNet50” helyett
THEN: az „Original” predikciók újraszámolódnak az új modellel; ha korábban már futott támadás, az „Adversarial” predikciók is frissülnek

- **Non-targeted támadás végrehajtása**

AS A: felhasználó I WANT TO: általános tévesztést előidézni a modellben

GIVEN: kép betöltve, „Targeted attack” jelölőnégyzet kikapcsolva, ϵ és iterációs szám csúszkák beállítva

WHEN: az „Attack” gombra kattint

THEN: a státuszsor „Running...” majd „Attack complete” üzenetet mutat, megjelenik a felnagyított perturbáció és a támadott kép, és az „Adversarial” predikciós panelen az eredeti top-1 osztály konfidenciája jellemzően csökken

- **Targeted támadás célosztály kereséssel**

AS A: felhasználó I WANT TO: egy konkrét osztályra rávenni a modellt

GIVEN: kép betöltve, a célosztály-kereső mezőbe elkezd gépelni egy osztálynevet

WHEN: a szűrt listából kiválaszt egy osztályt, majd az „Attack” gombra kattint

THEN: a „Targeted attack” jelölőnégyzet automatikusan bejelölődik, a státuszsor a kiválasztott osztály nevét mutatja, és a támadás után a célosztály konfidenciája nő

- **Epsilon csúszka élő előnézete FGSM módban**

AS A: felhasználó I WANT TO: interaktívan látni, hogyan változik a perturbáció erőssége

GIVEN: FGSM módszerrel már lefutott egy támadás (a gradiens előjele cache-elve van)

WHEN: az ϵ csúszkát új értékre húzza, PGD-re váltás nélkül

THEN: a perturbáció és az adversarial kép új gradiens-számítás *nélkül*, azonnal újraszámolódik a cache-elt előjel alapján

- **Attack gomb kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) támadást indítani kép betöltése előtt

GIVEN: az alkalmazás elindítva, nincs betöltött kép

WHEN: az „Attack” gombra kattint

THEN: egy „No image” információs párbeszédablak jelenik meg, a támadás nem indul el

- **Védekezési szűrő alkalmazása (Blur / JPEG)**

AS A: felhasználó I WANT TO: megvizsgálni, hogy egy egyszerű szűrő semlegesíti-e a zajt

GIVEN: egy sikeres támadás után az „Adversarial” kép elérhető

WHEN: a „Defend (Blur)” vagy „Defend (JPEG)” gombra kattint

THEN: az „Adversarial” képmező a szűrt képre frissül, és a predikciós panel az újraosztályozott (a szűrés után kapott) eredményeket mutatja

- **Defend gomb adversarial kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) védekezést alkalmazni támadás előtt

GIVEN: kép be van töltve, de még nem futott támadás

WHEN: valamelyik „Defend” gombra kattint

THEN: egy „No adversarial” információs párbeszédablak jelenik meg, semmilyen kép nem változik

- **Reset gomb**

AS A: felhasználó I WANT TO: új kísérletet kezdeni ugyanazzal a képpel

GIVEN: egy támadás és esetleg egy védekezés már lefutott

WHEN: a „Reset” gombra kattint

THEN: a perturbáció és az „Adversarial” képmező üresre vált, az „Adversarial” predikciós panel névtelen („—”) állapotba áll vissza, az „Original” kép és predikciói változatlanok maradnak

- **Hibakezelés támadás közben**

AS A: felhasználó I WANT TO: érthető visszajelzést kapni, ha a támadás valamiért megghiúsul

GIVEN: kép betöltve, a háttérszálon futó `_attack_worker` kivételt dob (pl. eszközhiba miatt)

WHEN: az „Attack” gombra kattint

THEN: egy „Attack failed” hibaablak jelenik meg a teljes veremkiíratással, a státuszszor „Attack failed.” szöveget mutat, az alkalmazás nem fagy le

5.5. Teszteredmények és értelmezésük

5.5.1. FGSM eredmények különböző ϵ értékekre

5.5.2. PGD eredmények különböző iterációs számokra

5.5.3. Targeted vs. non-targeted összehasonlítás

5.5.4. A védekezési modul hatásának mérése

5.6. A tesztelés korlátai

6. Összefoglalás

6.1. Elért eredmények

6.2. Következtetések

6.3. Továbbfejlesztési lehetőségek

Mellékletek

Hivatkozások

- [1] Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. arXiv:1412.6572, 2015.
- [2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. arXiv:1706.06083, 2018.